APPENDIX

PROOFS

Proof of Theorem 1 . Inspired by [75], we prove the NP-completeness by reducing the minimum vertex cover problem to the OSR problem.

Given a removal set I_N, whether $L(I \backslash I_N) > \beta$ can be verified in $O(mn^2)$. The time for verifying the non-conflict condition is $O(mn^2)$, and the time for checking minimality is $O(c^2)$. Therefore, whether a removal set I_N is a solution of the decision version of Problem 1 can be verified in polynomial time.

Consider a graph $G = (E, V)$, where $V = \{v_1, v_2, \ldots, v_{n_V}\}$ represents the set of vertices and $E = \{e_1, e_2, \ldots, e_{n_E}\}$ denotes the set of edges. This graph corresponds to a relation instance I with $R = \{E_1, E_2, \ldots, E_{n_E}, V, T, D\}$. Each edge $e_i = (v_1^i, v_2^i)$ gives rise to two tuples, t_1^i and t_2^i. For these tuples, we have $t_1^i[E_i] = t_2^i[E_i] = e$, $t_1^i[V] = g(v_1^i)$, and $t_2^i[V] = g(v_2^i)$. Additionally, $t_1^i[T] = u_1^i$ and $t_2^i[T] = u_2^i$. The remaining attribute values for t_1^i and t_2^i are set to 0. The function $g(\cdot)$ maps vertices $v \in V$ to positive numbers, such that $g(v_1^i) = id(v_1^i)b$ and $g(v_2^i) = id(v_2^i)b$. $id(v)$ are the index of $v \in V$. The inverse mapping $g^{-1}(\cdot)$ is defined as $g^{-1}(t_j^i[V]) = v_j^i$. The values u_j^i are computed as $u_1^i = (2i+1)^2 B$ and $u_2^i = (2i+2)^2 B$. Here, e, b, and B are positive numbers. The tuples induced by all $e_i \in E$ collectively form the set S_1.

A vertex $v_l \in V$ gives rise to a tuple t_l. For this tuple, we have $t_l[V] = t_l[T] = g(v_l)$, $t_l[D] = d$, and all other attribute values are set to 0. Here, $d > 0$ denotes a positive number. The collection of tuples derived from all $v_l \in V$ constitute the set S_2. Note that each vertex of v_l in this set is equal to some v_j^i vertices in S_1. These are the same vertices, represented differently. Furthermore, a tuple $t_j^i \in S_1$ corresponds to four additional tuples $t_{j1}^i, t_{j2}^i, t_{j3}^i, t_{j4}^i$, where $t_{jr}^i[V] = g(v_j^i) + r\triangle$ for $r = 1, 2, 3, 4$, and all other attributes are 0. The positive number $\triangle$ is a constant. The set of all such t_{jr}^i tuples is denoted as S_3. Each tuple $t_l \in S_2$ induces four tuples $t_{l1}, t_{l2}, t_{l3}, t_{l4}$. For these tuples, we have $t_{lr}[V] = g(v_l) + r\delta$ for $r = 1, 2, 3, 4$. The positive value $\delta > 0$ is a constant, and all other attributes are set to 0. This group of tuples $\forall t_{lr}$ forms the set S_4. The set S_5 comprises eight tuples, four of which contain only 0 values, and the other four contain only G values. The number G is sufficiently large. This set of tuples is employed to prevent distance normalization. The relationships between the constants are as follows:

$$\begin{aligned} &G > (2n_E+2)^2 B, (2n_E+2)b \\ &B, b \gg d \gg e \gg \triangle, \delta \ (\triangle \neq \delta) \\ &G - \frac{d}{2} > n_E \end{aligned} \tag{25}$$

The set $\Sigma = \{\varphi, \varphi_1, \cdots, \varphi_{n_E}\}$ contains DCs:

$$\varphi : \forall s, t \in I, \neg(s[V] = t[V], s[D] \neq t[D]), \tag{26}$$

$$\varphi_i : \forall s, t \in I, \neg(s[E_i] = e, t[E_i] = e, s[V] \neq t[V]), \tag{27}$$

$$(i = 1, \cdots, n_E)$$

With the DCs, the tuple pairs (t_1^i, t_2^i) do not satisfy φ_i, and the tuples t_l^i and t_l with $t_l^i[V] = t_l[V]$ violate φ. If we employ linear regression to train the dependency models and set $\gamma = 0$, $k = 1$, $\kappa = 3$, $L(t_j^i) = G - \frac{e}{2}$ for all $t_j^i \in S_1$ and $L(t_{jr}^i) = G$ for all $t_{jr}^i \in S_3$, the providers of t_j^i and t_{jr}^i are drawn from $\{t_{j1}^i, t_{j2}^i, t_{j3}^i, t_{j4}^i\}$. For $L(t_l) = G - \frac{d}{2}$ $(\forall t_l \in S_2)$ and $L(t_{lr}) = G$ $(\forall t_{lr} \in S_4)$, the providers of t_l and t_{lr} are from $\{t_{l1}, t_{l2}, t_{l3}, t_{l4}\}$. For all $t \in S_5$, $L(t) = G$ and the providers are from S_5. Hence, $L(t)$ for all $t \in I$ remains fixed during the repair process.

Suppose that $C > 0$ is a constant. We now prove that there exists a minimal removal set $I_N \subset I$ such that $L(I \setminus I_N) \geq -C(G - \frac{d}{2}) + (9n_E + 5n_V + 8)G - \frac{e}{2}n_E - \frac{d}{2}n_V$ if and only if there exists a vertex cover VC of G with a size of $|VC| \leq C$.

If $VC \subset V$ is a vertex cover of G with size $|VC| \leq C$, for each $t_j^i \in S_1$, if $g^{-1}(t_1^i[V]) \in VC$, then t_2^i is removed (regardless of whether $g^{-1}(t_2^i) \in VC$ or not), otherwise t_1^i is removed. In S_2, if $v_l \in VC$, then t_l is removed. Let the set of removed tuples be denoted as I_N. After deleting I_N, no conflicts remain. This is because one of the tuples in each $(t_1^i, t_2^i) \not\models \varphi_i$ is removed. The remaining tuple t_j^i could conflict with a t_l if $t_j^i[V] = t_l[V]$, but such t_l is also removed. It is evident that each removal is necessary, ensuring that I_N is minimal. As for $L(I \backslash I_N)$, the original $L(I) = (10n_E + 5n_V + 8)G - en_E - \frac{d}{2}n_V$. The decrease in L caused by removing I_N is greater than $n_E(G - \frac{e}{2}) + C(G - \frac{d}{2})$. Therefore, $L(I \backslash I_N) \geq -C(G - \frac{d}{2}) + (9n_E + 5n_V + 8)G - \frac{e}{2}n_E - \frac{d}{2}n_V$.

If I_N is a removal set of I with $L(I \setminus I_N) \geq -C(G - \frac{d}{2}) + (9n_E + 5n_V + 8)G - \frac{e}{2}n_E - \frac{d}{2}n_V$, there are C tuples $t_l \in S_2$ and n_E tuples $t_j^i \in S_1$ removed. This is because, in addition to the L decrease of $n_E(G - \frac{e}{2})$ caused by removing $t_j^i \in S_1$, the decrease of $C(G - \frac{d}{2})$ may be attributed to the removal of $t_l \in S_2$ or $t_j^i \in S_1$. Suppose that there are x t_l and $y + n_E$ t_j^i deleted, where

$$C - x = y\frac{G - \frac{e}{2}}{G - \frac{d}{2}} \tag{28}$$

The left-hand side of (28) is integral. If the equation holds, the right-hand side must also be integral. Since $\frac{G - \frac{e}{2}}{G - \frac{d}{2}}$ is fractional, to ensure the left-hand side is integral, y must be an integer multiple of $G - \frac{d}{2}$. However, given that $y \leq n_E < G - \frac{d}{2}$, the only scenario in which (28) can be satisfied is when $y = 0$ and $x = C$. Thus, the decrease of $C(G - \frac{d}{2})$ can only be attributed to the removal of C tuples $t_l \in S_2$. Suppose that VC is the set of v_l for $t_l \in I_N \cap S_2$. The size of VC is equal to the number of t_l removed, which is less than C. If VC is not a vertex cover of G, there exists an edge $e_i = (v_{l_1}, v_{l_2})$ that is not covered. So, both t_{l_1} and t_{l_2} would remain in S_2. While one of $t_1^i, t_2^i \in S_1$ remains $(g^{-1}(t_1^i[V]) = v_{l_1}, g^{-1}(t_2^i[V]) = v_{l_2})$, it would conflict with either t_{l_1} or t_{l_2}, which contradicts the non-conflict condition of $I \setminus I_N$. Therefore, VC must be a vertex cover with $|VC| \leq C$.

□

Proof of Proposition 2.

$$\sum_{t_l \in M(t_i)} \sum_{A_j \in R} \left(1 - \frac{|d_{ilj} - \hat{d}_{ilj}|}{G}\right)$$
$$\geq \sum_{t_l \in kNN(t_i)} \sum_{A_j \in R} \left(1 - \frac{|d_{ilj} - \hat{d}_{ilj}|}{G}\right)$$
$$\geq km\left(1 - \frac{C\varepsilon_M}{G}\right).$$

□

Proof of Proposition 3. Training $f_{lj}(D)$ minimizes

$$(d_{rlj} - f_{lj}(D_{rlj}; pars))^2.$$

If $\forall t_r \in kNN(t_l)$, $t_r[A_j] = t_l[A_j]$, then $d_{rlj} = 0$, and setting $f_{lj}(D) = 0$ yields zero training error. □

Proof of Proposition 4. If $t_l[A_j] = t_i[A_j]$ and $f_{lj}(D) = 0$, then

$$L'(t_i, t_l; A_j) = 1.$$

This occurs with probability $\eta\tau$. In all other cases, $L'(t_i, t_l; A_j) \geq 0$. Hence,

$$E(L'(t_i, t_l; A_j)) \geq \eta\tau.$$

Because $L'(t_i, t_l; A_j) \leq 1$, $E(L'(t_i, t_l; A_j)) \leq 1$. $E(L'(t_i)) = \sum_{t_l \in M(t_i)} \sum_{A_j \in R} E(L'(t_i, t_l; A_j)) \in [km\eta\tau, km]$. □

Proof of Proposition 5. For a $t_l \in I$, suppose that $S_l = |\{t_r | L(t_i, t_r) > L(t_i, t_l), t_r \in I \backslash I_C\}|$. Because $t_l \not\models \overline{M}(t_i)$, $S_l \geq k$. The smallest rank of t_l among $L(t_i, t_r)$ providers is $S_l + 1 > k$. Therefore, t_l can never provide $L(t_i, t_l)$ for t_i. □

Proof of Proposition 6. From (8),

$$L(t_i, t_l) = \sum_{A_j \in R} (G - \ell(t_i, t_l; A_j))\Gamma_i.$$

Given an instance I, suppose that we remove a set I_N to eliminate the conflicts. Then $\forall t_i \in I \backslash I_N$ has $u_i = 0$, as no conflict pair exists, indicating that $\Gamma_i = 1$. Therefore (19) is equivalent to,

$$\text{Max} \sum_{i=1}^{n} \sum_{t_l \in \overline{M}(t_i)} \sum_{A_j \in R} (G - \ell(t_i, t_l; A_j)) \tag{29}$$

If we set $\gamma = 0$, $\Gamma_i = 1$. The objective function is also in the form of (29). Therefore, if $L(t_i, t_l)$ is not fixed during repairing, it is equivalent to setting $\gamma = 0$.

□

Proof of Proposition 7. Constraint (12) ensures that at most one tuple remains for each pair $(t_i, t_l) \not\models \Sigma$ that does not satisfy Σ, thereby ensuring that the instance is non-conflict. The objective function (18), in conjunction with constraints (13) to (15), guarantees that $L(I \backslash I_N^*)$ is maximized. Regarding minimality, suppose that I_N^* is not a minimal removal set. Then there exists a tuple $t_i \in I_N^*$ such that for all $t_l \in I \setminus I_N^*$, the pair $(t_i, t_l) \models \Sigma$. For any $t_l \in I \setminus I_N^*$, let its conformance in $I \setminus I_N^*$ and $I \setminus (I_N^* \setminus t_i)$ be denoted as $L(t_l | I \setminus I_N^*)$ and $L(t_l | I \setminus (I_N^* \setminus t_i))$, respectively. The following relationship holds:

$$L(t_l | I \backslash I_N^*) \leq L(t_l | I \backslash (I_N^* \backslash \{t_i\})). \tag{30}$$

Besides, $L(t_i | I \backslash (I_N^* \backslash \{t_i\})) > 0$. So putting back t_i into $I \backslash I_N^*$ makes

$$L(I \backslash I_N^*) \leq L(I \backslash (I_N^* \backslash \{t_i\})), \tag{31}$$

contradicting $L(I \backslash I_N^*)$ being maximum. □

Proof of Proposition 8. For a given LP formulation (19), x_i, y_{il} represent the values of decision variables, and a solution $X = [x_i, y_{il}]$ is a vector consisting of x_i, y_{il}. The solution X is considered feasible if it adheres to all constraints (12) to (15). The feasible region of the LP is the set of all feasible solutions X, and an extreme point is one of its vertices. When the value of x_i is changed, the values of other variables $x_l, y_{il} \in X$ are correspondingly adjusted to yield a new optimal solution $X' = [x_i', y_{il}']$. The change in the objective value due to the modification of x_i is calculated as $\sum_{t_i \in I} \sum_{t_l \in \overline{M}(t_i)} L(t_i, t_l)(y_{il}' - y_{il})$.

By analyzing the solution space of LP, we find a necessary condition of the solution X returned by LP.

Proposition (Necessary Condition for LP Solution)**.** *Suppose that X is a feasible solution. X_P and X_N are disjoint subsets of X. For any element x_l within X, the set $\{y_{l1}, y_{l2}, \ldots, y_{ls_l}\}$ encompasses all y_{lr} values greater than zero, and s_l denotes its size. Consider $\varepsilon > 0$, an arbitrarily small constant. $X^+ = [x_i + \varepsilon, x_l - \varepsilon, x_r]$ and $X^- = [x_i - \varepsilon, x_l + \varepsilon, x_r]$ are two induced solutions, where $x_i \in X_P$, $x_l \in X_N$, and $x_r \in X \backslash (X_P \cup X_N)$. The solution X can be turned into X^+ if X^+ is feasible, and similarly for X^-.*

If X is returned, for $\forall X_P, X_N \subset X$, there exists $x_l \in X$, either turning X to X^+ or X^- makes $kx_l - \sum_{r=1}^{s_l - 1} y_{lr} > x_{s_l}$.

Proof. Proof by Contradiction: Suppose that when X is identified as the solution to the linear program (LP), there exist $X_P, X_N \subset X$ such that transforming X to X^+ and X^- results in no $x_l \in X$ satisfying the inequality $kx_l - \sum_{r=1}^{s_l - 1} y_{lr} > x_{s_l}$. The set X can be expressed as the convex combination $X = 0.5X^+ + 0.5X^-$, indicating that it is not an extreme point of the feasible region. This contradicts the assertion that X is the solution returned by the LP. If no X_N, X_P exists, X cannot be represented as any convex combination of other feasible solutions X^+, X^-. So X is an extreme point.

Note: When altering the values of x_i to $x_i + \varepsilon$ and x_l to $x_l - \varepsilon$ (where $x_i \in X_P$ and $x_l \in X_N$), the values of y_{ir} and y_{rl} are correspondingly adjusted simultaneously. If $y_{ir} = x_i$, then y_{ir} undergoes the same modification as x_i, and similarly for the case where $y_{ir} = x_r$, $y_{lr} = x_l$. The y_{lr} terms in the inequality $kx_l - \sum_{r=1}^{s_l - 1} y_{lr} > x_{s_l}$ represent the values after these modifications. □

This proposition indicates that a necessary condition for a feasible solution of the LP is that altering the value of any X_P, X_N results in some x_l satisfying $kx_l - \sum_{r=1}^{s_l - 1} y_{lr} > x_{s_l}$. As stated in [76], any solution returned by an LP model is an

extreme point of its feasible region. In our LP formulation, there are two types of extreme points.

Definition ($\mathcal{X}$-solution, $\mathcal{Y}$-solution)**.** *A feasible solution X is an $\mathcal{X}$-solution iff $\{x_i\} \subset X$ forms an extreme point of the feasible region of (12). Otherwise, it is a $\mathcal{Y}$-solution.*

Accordingly, an $\mathcal{X}$-solution relies on $x_i \in X$ to form an extreme point of (12), and a $\mathcal{Y}$-solution is determined by $y_{lr} \in X$ to meet $kx_l - \sum_{r=1}^{s_l-1} y_{lr} = x_{ls_l}$. It is evident that **the LP result is either an $\mathcal{X}$-solution or a $\mathcal{Y}$-solution.**

In practice, we find that the LP could seldom return a $\mathcal{Y}$-solution, so we further analyze the precondition of $\mathcal{Y}$-solution.

Proposition (Necessary Condition for $\mathcal{Y}$-Solution)**.** *Suppose that $X = [x_i, y_{lr}]$ is a $\mathcal{Y}$-solution. Then,*

(1) $\exists X_P, X_N \subset X$, such that both X^+ and X^- satisfy constraints (12).

(2) Consider X^+. Suppose that $x_i^+, y_{lr}^+ \in X^+$ and

$$\begin{aligned} T^+ = \{t_l \mid & (kx_l^+ - \sum_{r=1}^{s_l-1} y_{lr}^+) - (kx_l - \sum_{r=1}^{s_l-1} y_{lr}) > 0, \\ & kx_l - \sum_{r=1}^{s_l-1} y_{lr} = x_{s_l}\}, \\ T^- = \{t_l \mid & (kx_l^+ - \sum_{r=1}^{s_l-1} y_{lr}^+) - (kx_l - \sum_{r=1}^{s_l-1} y_{lr}) < 0, \\ & kx_l - \sum_{r=1}^{s_l-1} y_{lr} = x_{s_l}\}. \end{aligned} \tag{32}$$

Let $\triangle y_{lr} = y_{lr}^+ - y_{lr}$ be the value change of y_{lr} caused by changing X to X^+. Set

$$\begin{aligned} \alpha = & \sum_{t_l \in I} \sum_{r=1}^{s_l-1} \triangle y_{lr} L(t_l, t_r) + \sum_{t_l \in I \setminus (T^- \cup T^+)} \triangle y_{lr} L(t_l, t_{s_l}) \\ & + \sum_{t_l \in (T^+ \cup T^-)} (kx_l - \sum_{r=1}^{s_l-1} y_{lr}) L(t_l, t_{s_l}), \end{aligned}$$

If X is returned as a $\mathcal{Y}$-solution, for X^+ obtained from arbitrary X_P and X_N, the following two formulas are satisfied.

$$\alpha - \sum_{t_l \in T^+} (\triangle y_{ls_l} L(t_l, t_{s_l}) - \triangle y_{l,s_l+1} L(t_l, t_{s_l+1})) \leq 0 \tag{33}$$

$$-\alpha - \sum_{t_l \in T^-} (\triangle y_{ls_l} L(t_l, t_{s_l}) - \triangle y_{l,s_l+1} L(t_l, t_{s_l+1})) \leq 0 \tag{34}$$

Proof. For condition (1), if for all $X_N, X_P \subset X$, either X^+ or X^- violates constraint (12), then X cannot be expressed as a convex combination of any feasible solutions. Consequently, $x_i \in X$ constitutes an extreme point of the feasible region defined by (12), which is incongruent with the definition of a $\mathcal{Y}$-solution.

For condition (2), if (33) does not hold, then X^+ is a feasible solution that yields a higher objective value than X. This contradicts the premise that X is the optimal solution. □

(33) and (34) form the necessary condition of $\mathcal{Y}-$solution, if

$$|\alpha| \geq |\sum_{t_l \in T^+} (\triangle y_{ls_l} L(t_l, t_{s_l}) - \triangle y_{l,s_l+1} L(t_l, t_{s_l+1}))|, \tag{35}$$

and

$$|\alpha| \geq \sum_{t_l \in T^-} (\triangle y_{ls_l} L(t_l, t_{s_l}) - \triangle y_{l,s_l+1} L(t_l, t_{s_l+1})), \tag{36}$$

no $\mathcal{Y}-$solution can be returned by the LP.

Because

$$\begin{aligned} |\alpha| \geq & \sum_{t_l \in I} (L(t_l, t_{s_l-1}) \cdot \sum_{r=1}^{s_l-1} \triangle y_{lr}), \\ & |\sum_{t_l \in T^+ \subset I} (\triangle y_{ls_l} L(t_l, t_{s_l}) - \triangle y_{l,s_l+1} L(t_l, t_{s_l+1}))| \\ \leq & \sum_{t_l \in T^+ \subset I} (L(t_l, t_{s_l}) \cdot \sum_{r=1}^{s_l-1} \triangle y_{lr}), \\ \leq & \sum_{t_l \in I} (L(t_l, t_{s_l-1}) \cdot \sum_{r=1}^{s_l-1} \triangle y_{lr}), \end{aligned}$$

and

$$\begin{aligned} & |\sum_{t_l \in T^- \subset I} (\triangle y_{ls_l} L(t_l, t_{s_l}) - \triangle y_{l,s_l+1} L(t_l, t_{s_l+1}))| \\ \leq & \sum_{t_l \in T^- \subset I} (L(t_l, t_{s_l}) \cdot \sum_{r=1}^{s_l-1} \triangle y_{lr}) \\ \leq & \sum_{t_l \in I} (L(t_l, t_{s_l-1}) \cdot \sum_{r=1}^{s_l-1} \triangle y_{lr}), \end{aligned}$$

the relations in (35) and (36) hold. Therefore, no $\mathcal{Y}-$solution can be returned.

□

Proof of Proposition 9. X is an $\mathcal{X}$-solution, which implies that $x_i \in X$ constitutes an extreme point of the feasible region defined by (12). If we consider each conflict tuple t_i as a vertex and each conflict pair (t_i, t_l) as an edge, then (12) defines the constraints for the minimum vertex cover problem. As stated in [76], the feasible region of (12) is half-integral, meaning that for all $x_i \in X$, $x_i \in \{0, 0.5, 1\}^n$. Suppose there exists a y_{il} not in $\{0, 0.5, 1\}^n$. Then, there exists $x_i > y_{il}$ and $x_l > y_{il}$. By turning $y_{il} \notin \{0, 0.5, 1\}$ with $max_{t_l \in \overline{M}(t_i)} L(t_i, t_l)$ into $min\{x_i, x_l\}$ could get a solution with a larger objective. Therefore, a solution with y_{il} not in $\{0, 0.5, 1\}$ isn't optimal. □

Proof of Proposition 10. Because in an integral feasible solution X, at most one tuple is left in each I_q, indicating $\sum_{t_i \in I_q} x_i \leq 1$, the clique constraints added into the ILP are redundant constraints. Therefore the solution space of ILP is not impacted. □

Proof of Proposition 11. Once all clique constraints for each maximal clique are incorporated and the LP yields an $\mathcal{X}$-

solution, we are concerned only with the feasible region of the relaxed problem (P').

$$(P'): x_i + x_l + u_{il} = 1. \quad \sum_{t_r \in I_q} x_r + u_{I_q} = 1 \tag{37}$$

Here, u_{il} and u_{I_q} serve as variables for constraint standardization. The constraints $x_i + x_l + u_{il} = 1$ are introduced for conflict pairs (t_i, t_l) where t_i and t_l do not share any common cliques. The solution that includes u_{il}, u_{I_q} is represented as $X' = [x_i, u_{il}, u_{I_q}]$. All constraints can be expressed in an alternative form:

$$AX'^T = \mathbf{1}. \tag{38}$$

In this equation, A denotes the parameter matrix, and $\mathbf{1}$ is a vector of all ones. The column vectors A_i, A_{il}, and A_{I_q} correspond to the variables x_i, u_{il}, and u_{I_q}, respectively. According to [77], if X (or X') is an extreme point of (P) (or (P')), the column vectors of the positive variables in X' are linearly independent. Consequently, for any extreme point X' of (P'), the number of positive variables must be less than the number of constraints in (P'). In a chain, there is at least one positive variable (either x_i, x_l, or u_{il}) in each constraint. If an equation $x_i + x_l + u_{il} = 1$ contains two positive variables, their count exceeds the number of constraints. Similarly, within a clique constraint $\sum_{t_i \in I_q} x_i + u_{I_q} = 1$, if there are more than one positive variable, their number will also exceed the number of constraints. □

Proof of Proposition 12. A clique may be identified with $x_i = 0.5$ when there exists a clique $I_q = \{t_1, t_2, t_3\}$ of size 3 that has not been limited by a clique constraint. In the worst case, only one such size-3 clique can be detected per iteration. Following C_c^3 iterations, all potential size-three cliques will be encompassed by at least one clique constraint, so the loop terminates. □

Proof of Proposition 13. The minimal condition is ensured by Lines 16 to 18 in Alg.1, as any redundantly removed tuples are put back to $I \backslash I_N$. The time complexity of $O(mn^2)$ accounts for error detection and the computation of $L(t_i, t_l)$, which can be performed offline. The complexity of $O((Kn + c)^{3.5})$ corresponds to the execution time of the LP solver. The time complexity of $O(c^3)$ is associated with the convergence process. The time required for checking minimality is $O(c \log(c) + c^2)$. Therefore, the overall complexity is $O(mn^2 + (nK + c)^{3.5} c^3)$.

□

Proof of Proposition 14. The size of $I \backslash I_N$ compared to $I \backslash I_N^*$ is $\frac{|I \backslash I_N|}{|I \backslash I_N^*|} \geq \frac{|I \backslash I_C|}{|I|}$. For each $t_i \in I \backslash I_N$, $\frac{L(t_i | I \backslash I_N)}{L(t_i | I \backslash I_N^*)} \geq \frac{k \min L(t_i, t_l)}{k \max L(t_i, t_l)} = \eta$. So the error bound of Alg.1 is $\frac{L(I \backslash I_N)}{L(I \backslash I_N^*)} \geq \eta \frac{|I \backslash I_C|}{n}$. □

Proof of Proposition 15. Case 1: $t_l \in M(t_i)$ means that t_l is one of the top-k $L(t_i, t_r)$ providers of t_i. $t_i, t_l \in I \backslash I_C$ means that t_i, t_l can not be removed during repair. To maximize the objective (19), $y_{il} = 1$.

Case 2: $t_i \in I_C$ and $t_l \in I \backslash I_C$, means that $y_{il} \leq x_i$ and $y_{il} \leq 1$. y_{il} is determined by x_i. $t_l \in M(t_i)$ also means that y_{il} should be 1 to maximize the objective. (Case 3 is the same)

Case 4: $(t_i, t_l) \not\models \Sigma$, indicates at least one of t_i, t_l should be removed, i.e., $x_i = 0$ or $x_l = 0$. With $y_{il} \leq x_i$ and $y_{il} \leq x_l$, we have $y_{il} \leq min(x_i, x_l) = 0$. (Case 5 is the same) □

Proof of Proposition 16. The minimal condition is safeguarded by Line 5 to Line 7 in Alg.2. The computational time for error detection and the evaluation of $L(t_i, t_l)$ is $O(mn^2)$. The time complexity for tuple removal is $O(n^2)$, while the complexity for the minimality check is $O(c \log(c) + c^2)$. So the overall complexity of the algorithm is dominated by the error detection and $L(t_i, t_l)$ calculation, resulting in a total complexity of $O(mn^2)$. □

Proof of Proposition 17. The expectation of $L(I \backslash I_N)$ can be calculated as,

$$E(L(I \backslash I_N)) = \sum_{t_i \in I} P_i \sum_{t_l \in \overline{M}(t_i)} P_l P_{il}^{in} L(t_i, t_l) \tag{39}$$

$P_i = \prod_{(t_l, t_i) \not\models \Sigma} P_{il}$ and $P_l = \prod_{(t_r, t_l) \not\models \Sigma} P_{lr}$ are the remaining probability of t_i and t_l. P_{il}^{in} is the probability of t_l providing $L(t_i, t_l)$ for t_i after repair. For $t_l \in M(t_i)$, $P_{il}^{in} = 1$ because they provide top-k $L(t_i, t_l)$ at the beginning. While actually, they cannot provide $L(t_i, t_l)$ if they are removed. Such probability is controlled by P_l. To find the error bound,

$$\begin{aligned} E(L(I \backslash I_N)) &\geq (\frac{\eta}{1+\eta})^{2V} min L(t_i, t_l) \sum_{t_i \in I} \sum_{t_l \in \overline{M}(t_i)} P_{il}^{in} \\ &\geq (\frac{\eta}{2})^{2V} min L(t_i, t_l) nk \end{aligned} \tag{40}$$

Combining with $L(I \backslash I_N^*) \leq max L(t_i, t_l) nk$,

$$\frac{E(L(I \backslash I_N))}{L(I \backslash I_N^*)} \geq (\frac{\eta}{2})^{2V+1}. \tag{41}$$

□

CELL-LEVEL ESTIMATION

Because S-repair often serves as a subroutine in U-repair methods, we combine the S-repair methods (TE-LP, OptSRepair, Soft, Relaxation, Probabilistic, Clique) with U-repair methods (Statistical, Raha+Baran [32], NADEEF, SCARE, Holistic, HC, Horizon). Suppose I_N is the set of tuples removed by an S-repair method. For Statistical and SCARE, which rely on a consistent tuple set for cleaning, we directly set I_N as the consistent set. For Raha+Baran, we double the weight of tuples in I_N to increase their likelihood of being selected for labeling. For Holistic, we halve the cost of cells in I_N to make them more likely to be cleaned. For Horizon, we halve the pattern quality of the tuples in I_N. For HC and NADEEF, we halve the weight of the original values for the cells in I_N. The results are shown in Table IV.

Table IV
Update repairs on Flights and Rayyan

Methods	Flights			Rayyan		
	Pre	Rec	F1	Pre	Rec	F1
NADEEF	0.174	0.149	0.161	0.000	0.000	0.000
SCARE	0.169	0.298	0.216	0.000	0.000	0.000
Statistical	0.682	0.332	0.447	0.000	0.000	0.000
Holistic	0.582	0.282	0.380	0.475	0.121	0.193
HC	**0.991**	0.041	0.079	0.000	0.000	0.000
Raha+Baran	0.767	0.219	0.451	**0.603**	**0.301**	**0.401**
Horizon	0.663	**0.372**	**0.476**	0.000	0.000	0.000
Statistical+TE-LP	**0.822**	0.276	0.413	0.000	0.000	0.000
Statistical+OptSRepair	0.809	0.257	0.391	0.000	0.000	0.000
Statistical+Soft	0.593	0.275	0.375	0.000	0.000	0.000
Statistical+Relaxation	0.724	0.322	0.446	0.000	0.000	0.000
Statistical+Probabilistic	0.688	0.369	0.479	0.000	0.000	0.000
Statistical+Clique	0.689	**0.388**	**0.496**	0.000	0.000	0.000
Raha+Baran+TE-LP	0.665	0.360	0.467	**0.613**	0.287	0.391
Raha+Baran+OptSRepair	0.741	0.246	0.369	0.602	0.256	0.359
Raha+Baran+Soft	0.654	0.262	0.374	0.576	0.214	0.312
Raha+Baran+Relaxation	0.739	0.355	0.480	0.592	0.302	0.400
Raha+Baran+Probabilistic	0.796	0.387	0.519	0.588	0.308	0.404
Raha+Baran+Clique	0.801	**0.391**	**0.525**	0.591	**0.311**	**0.408**
NADEEF+TE-LP	**0.183**	0.153	0.167	0.000	0.000	0.000
NADEEF+OptSRepair	0.171	0.146	0.158	0.000	0.000	0.000
NADEEF+Soft	0.155	0.121	0.136	0.000	0.000	0.000
NADEEF+Relaxation	0.152	0.156	0.154	0.000	0.000	0.000
NADEEF+Probabilistic	0.169	0.177	0.173	0.000	0.000	0.000
NADEEF+Clique	0.170	**0.182**	**0.176**	0.000	0.000	0.000
SCARE+TE-LP	0.171	0.317	0.222	0.000	0.000	0.000
SCARE+OptSRepair	0.170	0.302	0.217	0.000	0.000	0.000
SCARE+Soft	0.174	0.301	0.221	0.000	0.000	0.000
SCARE+Relaxation	0.176	0.335	0.231	0.000	0.000	0.000
SCARE+Probabilistic	0.179	0.337	0.233	0.000	0.000	0.000
SCARE+Clique	**0.180**	**0.343**	**0.236**	0.000	0.000	0.000
Holistic+TE-LP	0.595	0.285	0.385	**0.481**	0.119	0.191
Holistic+OptSRepair	0.574	0.322	0.413	0.446	0.129	0.200
Holistic+Soft	0.592	0.246	0.348	0.458	0.126	0.198
Holistic+Relaxation	0.604	0.336	0.432	0.469	0.168	0.247
Holistic+Probabilistic	**0.617**	**0.385**	**0.474**	0.472	0.196	0.277
Holistic+Clique	**0.617**	**0.385**	**0.474**	0.477	**0.198**	**0.280**
HC+TE-LP	0.982	0.061	0.115	0.000	0.000	0.000
HC+OptSRepair	0.975	0.054	0.102	0.000	0.000	0.000
HC+Soft	**0.993**	0.041	0.079	0.000	0.000	0.000
HC+Relaxation	0.928	0.079	0.146	0.000	0.000	0.000
HC+Probabilistic	0.940	0.084	0.154	0.000	0.000	0.000
HC+Clique	0.942	**0.087**	**0.159**	0.000	0.000	0.000
Horizon+TE-LP	**0.665**	0.369	0.475	0.000	0.000	0.000
Horizon+OptSRepair	0.644	0.352	0.455	0.000	0.000	0.000
Horizon+Soft	0.638	0.322	0.428	0.000	0.000	0.000
Horizon+Relaxation	0.656	0.384	0.484	0.000	0.000	0.000
Horizon+Probabilistic	0.651	0.428	0.517	0.000	0.000	0.000
Horizon+Clique	0.655	**0.429**	**0.521**	0.000	0.000	0.000

From the results, we observe that when our methods are used as subroutines in these U-repair methods, there is a notable improvement in accuracy. Specifically, we find that our methods significantly increase the recall of these U-repair methods, with only a small impact on precision. This demonstrates that by more accurately identifying erroneous tuples, U-repair methods are better able to locate erroneous cells and achieve more accurate repairs.

Besides, we find that several U-repair methods perform poorly on Rayyan, which has an extremely low repetition rate. This highlights the limitation of existing repair approaches that rely on the most frequent patterns, such as Horizon. These methods suffer from insufficient pattern support and fail to effectively repair the data. A similar issue arises with probability-based methods like SCARE and distribution-based methods like Statistical, where the joint distribution of values is highly sparse, leading to poor performance in such cases. HC relies on co-occurrence to build candidate values for potentially erroneous cells. Due to the low repetition rate, the sets of candidate values are mostly empty sets, indicating that a cleaning system integrating various signals may also be dragged down by certain signals.

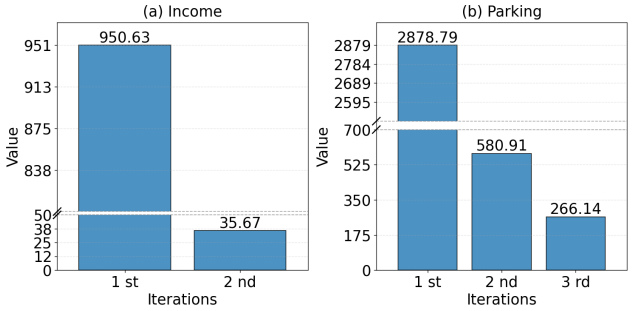

Fig. 10: Time of each iteration on Income and Parking

Effectiveness of Acceleration Techniques

TABLE VII: Variable Reduction Rate of the First Iteration

Dataset	Flights	Rayyan	Company	Soccer
Reduction Rate	0.322	0.305	0.289	0.139
Dataset	Restaurant	Inspection	Income	Parking
Reduction Rate	0.799	0.377	0.807	0.361
Dataset	Iris	Yeast	AirQuality	SocialMedia
Reduction Rate	0.115	0.452	0.396	0.122

We estimate the variable reduction rate in Table VII and time of each iteration in Figure 10. We can find that, our speeding techniques **Case 1-4** can reduce over 30% variables in the first iteration. Because of **Case 5**, the running speed of each iteration decreases compared to the former one. Both of the results show the effectiveness of our acceleration techniques.

Estimation of Amount and Type of Models

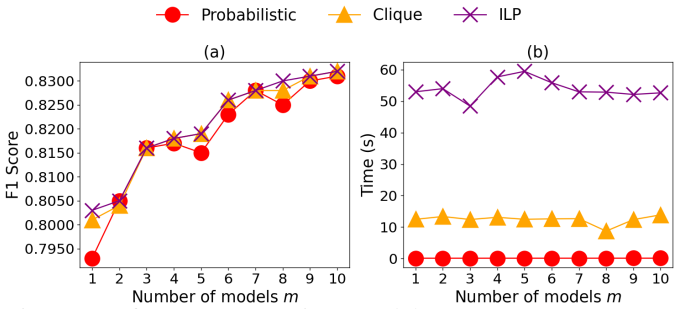

Fig. 11: Performance on various model amount m over Rayyan

Figure 11 shows the effect of varying the number of models m trained for each tuple t_i in dataset Rayyan. As m increases from 1 to 10, accuracy improves, suggesting that more models capture attribute dependencies better. In Figure 11 (b), the time

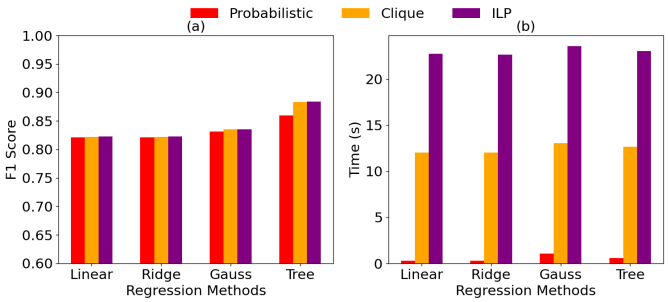

Fig. 12: Performance on regression models over Rayyan

cost remains stable across methods. Notably, Clique and ILP yield similar results in most cases, highlighting the optimality of Clique. Figure 12 shows performance with different regression models (Linear, Ridge, Gaussian, and Decision Tree). As model capability increases, repair ability improves, but more time is spent on training and prediction.

EXAMPLES OF OUR METHOD

Example 3. Figure 1a shows the 2024 monthly household electricity bill in Shanghai, including Room Number, Month, Temperature (average monthly temperature), Usage, and Charge. Tuples with the same Month should have the same Temperature, and larger Usage should correspond to higher Charges. There exist several inconsistencies. Tuples t_{10}, t_{11}, and t_{12} share the same Month but have very different Temperatures; t_5 and t_7 have different Usages but identical Charges. In December, the average temperature in Shanghai is around 5.7°C [60], so t_{10} and t_{12}, with Temperatures above 20°C, are likely errors. Most average electricity charges (Figure 1b) fall between 0.62 and 0.75, while t_5 is 0.90, suggesting an error. Thus, $\{t_5, t_{10}, t_{12}\}$ is a reasonable removal set. Although this approach can detect the errors, it relies on domain knowledge, which may not always be available.

To solve such conflicts in an automated way, there exist several data cleaning strategies that can be applied.

The first technique is the minimum principle, i.e., removing a minimum set of tuples to eliminate conflicts. However, in this example, there exist six possible minimum removal sets: $\{t_5, t_{10}, t_{11}\}$, $\{t_5, t_{10}, t_{12}\}$, $\{t_5, t_{11}, t_{12}\}$, $\{t_7, t_{10}, t_{11}\}$, $\{t_7, t_{10}, t_{12}\}$, and $\{t_7, t_{11}, t_{12}\}$. Without considering attribute values, a method based solely on the minimum principle may select an incorrect removal, such as $\{t_5, t_{10}, t_{11}\}$.

Aggregation-based methods assign higher confidence to tuples close to others, e.g., maximum density [9] (Figure 1c). Tuples with lower density are considered less reliable, so the removal set $\{t_7, t_{10}, t_{12}\}$ is selected. However, t_7 is actually reasonable despite its low density, showing that aggregation-based methods can fail when the data distribution is sparse.

Co-occurrence-based methods repair data by favoring the most frequent value combinations. However, as shown in Figure 1a, the repetition rate is low and each value combination appears only once, making it difficult to identify a reliable removal set using this strategy.

id	Room Number	Month	Temperature	Usage	Charge
t_{13}	RM2-601	November	12.0	296	184.96
t_{14}	RM2-501	November	12.0	291	181.84
t_{15}	RM2-502	November	15.0	285	178.09

Fig. 13: 2024 monthly household electricity bill ($t_{13} - t_{15}$)

When applying a cleaning system such as HoloClean (HC) [7], which leverages co-occurrence to generate candidate values, the same issue arises. Due to the low repetition rate, co-occurrence fails to provide alternative candidates, and all candidate sets retain the original values, resulting in no modification. This demonstrates that ineffective signals may degrade the performance of an integrated cleaning system.

Overall, relying solely on the minimum principle, aggregation, or co-occurrence is insufficient. Therefore, we study attribute dependencies to better capture relationships between attributes and to identify a more reasonable removal set, namely $\{t_5, t_{10}, t_{12}\}$.

Example 4. Consider Figure 1a. A DC is defined as:

$$\begin{aligned}\varphi_1 : \forall t_i, t_l \in I, \neg(t_i[Month] = t_l[Month] \\ \wedge\, t_i[Temperature] \neq t_l[Temperature]),\end{aligned} \tag{42}$$

meaning that for any tuple pairs $t_i, t_l \in I$, if they satisfy $t_i[Month] = t_l[Month]$ and $t_i[Temperature] \neq t_l[Temperature]$, they are involved in a conflict. Similarly, for the relations between $Usage$ and $Charge$, we can define:

$$\begin{aligned}\varphi_2 : \forall t_i, t_l \in I, \neg(t_i[Usage] > t_l[Usage] \\ \wedge\, t_i[Charge] \leq t_l[Charge]).\end{aligned} \tag{43}$$

The set of DCs in Figure 1a is $\Sigma = \{\varphi_1, \varphi_2\}$. In Figure 1a, (t_{10}, t_{11}), (t_{10}, t_{12}), (t_{11}, t_{12}) violate φ_1 and (t_5, t_7) violates φ_2. Among the tuples in conflict, $I_N = \{t_5, t_{10}, t_{11}\}$ is a minimal removal set because putting back any $t_i \in I_N$ makes at least one conflict left.

Note: A minimal removal set is distinct from a minimum removal set. According to Figure 2, (t_{13}, t_{15}) and (t_{14}, t_{15}) violate φ_1. $\{t_{15}\}$ is a minimum removal set, as it has the smallest size. However, both $s_1 = \{t_{13}, t_{14}\}$ and $s_2 = \{t_{15}\}$ are minimal, because putting back any tuple from either s_1 or s_2 would leave a conflict in the remaining table. Avoiding over-repair is a widely accepted consensus in data cleaning, and minimal condition is the approach we use to achieve this.

Example 5. In Figure 1a. Suppose that the size of the training set $\kappa = 4$. To obtain f_{15}, the 4−nearest non-conflict tuples t_2, t_4, t_6, t_9 are leveraged. First, we calculate the distance vector between t_i and t_l ($i, l \in \{1, 2, 4, 6, 9\}$, $i < l$).

$$\begin{aligned}D_{125} &= [1.000, 0.352, 0.421, 0.037, 0.025],\ d_{125} = 0.032; \\ D_{145} &= [1.000, 0.250, 0.696, 0.222, 0.706],\ d_{145} = 0.751; \\ &\cdots \\ D_{695} &= [1.000, 0.353, 0.600, 0.148, 0.179],\ d_{695} = 0.156;\end{aligned} \tag{44}$$

Fit f_{15} with

$$f_{15}: \begin{bmatrix} D_{125} \\ D_{145} \\ \cdots \\ D_{695} \end{bmatrix} \rightarrow \begin{bmatrix} d_{125} \\ d_{145} \\ \cdots \\ d_{695} \end{bmatrix}. \tag{45}$$

For any tuple t_i within the set I, f_{15} can be utilized to predict the discrepancy between $t_1[Charge]$ and $t_i[Charge]$. This estimation is denoted as $\hat{d}_{i15}$. Suppose that t_2 is the given t_i. The distance vector:

$$\begin{aligned} D_{215} &= [1, d_{211}, d_{212}, d_{213}, d_{214}] \\ &= [1.000, 0.352, 0.421, 0.037, 0.025]. \end{aligned} \tag{46}$$

Then the estimated distance $\hat{d}_{215}$ is calculated as

$$\hat{d}_{215} = f_{15}(D_{215}) = 0.029. \tag{47}$$

Example 6. In Figure 1a, let us consider a minimal removal set $I_N = \{t_5, t_{10}, t_{12}\}$. With $k = 2$ and $\gamma = 0.2$, the conformance of the remaining dataset $I \backslash I_N$ can be computed using Equation (8), resulting in $L(I \backslash I_N) = 10.023$. By evaluating the conformance for all possible removal sets I_N, we obtain the $L(I \backslash I_N)$ values for each corresponding remaining set. For instance, when $I_N = \{t_5, t_{10}, t_{11}\}$, the conformance is $L(I \backslash I_N) = 9.458$, and when $I_N = \{t_7, t_{11}, t_{12}\}$, $L(I \backslash I_N) = 8.532$. After calculating $L(I \backslash I_N)$ for all potential removal sets, the set $\{t_5, t_{10}, t_{12}\}$ yields the highest $L(I \backslash I_N)$ value, indicating the best conformance to the attribute dependencies.

Example 7. Consider the relation instance in Figure 1a. First we find $\overline{M}(t_i)$ for each $t_i \in I$ ($k = 2$). For t_1, we could calculate $L(t_1, t_{10}) = 0.575$, $L(t_1, t_{11}) = 0.447$, $L(t_1, t_{12}) = 0.437$, $L(t_1, t_2) = 0.375$, $L(t_1, t_8) = 0.027$, $\cdots$. Among all potential providers, tuples t_{10}, t_{11}, t_{12} belong to the conflict set I_C, while t_2, t_8 are non-conflict. For any tuple t_l where $L(t_1, t_l) < L(t_1, t_8)$, it is impossible for t_l to serve as a provider for t_1. So $\overline{M}(t_1) = \{t_{10}, t_{11}, t_{12}, t_2, t_8\}$.

Next, we define variables $x_5, x_7, x_{10}, x_{11}, x_{12}$ to represent the removal of tuples $t_5, t_7, t_{10}, t_{11}, t_{12}$. The variables y_{il} indicate if t_l is an $L(t_i, t_l)$ provider after the repair, for all $t_i \in I$ and $t_l \in \overline{M}(t_i)$. To ensure no conflicts, we impose constraints like $x_5 + x_7 \leq 1$ on conflicting pairs and set $y_{57} \leq x_5$ and $y_{57} \leq x_7$ to ensure $y_{il} = 1$ is only possible when both t_i and t_l are not removed. For non-conflicting tuples, constraints like $\sum_{t_l \in \overline{M}(t_1)} y_{1l} \leq 2$ are used, and for conflicting tuples, constraints like $\sum_{t_l \in \overline{M}(t_5)} y_{5l} \leq 2x_5$ limit the number of providers. The ILP solution gives $x_5 = x_{10} = x_{12} = 0$ and $x_7 = x_{11} = 1$, so the optimal $I_N = \{t_5, t_{10}, t_{12}\}$.

Example 8. Consider Figure 1a, where the tuple set $I_q = \{t_{10}, t_{11}, t_{12}\}$ constitutes a clique. Furthermore, it is a maximal clique because tuples $t_i \notin I_q$ do not conflict with those in I_q. Assume that the solution obtained from the LP relaxation is $x_{10} = x_{11} = x_{12} = 0.5, x_5 = 0, x_7 = 1$. To tighten the constraints, a clique constraint $x_{10} + x_{11} + x_{12} \leq 1$ is incorporated into (12). Note that t_5, t_7 form a chain, and I_q is a disjoint clique. According to Proposition 11, if the LP yields an $\mathcal{X}$-solution X after including the constraint $x_{10}+x_{11}+x_{12} \leq 1$, then $I_N = \{t_i | x_i = 0, x_i \in X\}$ represents the exact solution to the ORS problem.

Example 9. Consider Figure 1a. Algorithm 1 addresses the LP relaxation of the ILP from Example 7 during the first iteration (Lines 3-7). Suppose the LP solution includes $x_{10} = x_{11} = x_{12} = 0.5$, $x_5 = 0$, and $x_7 = 1$. We construct $I_h = \{x_{10}, x_{11}, x_{12}\}$ (Line 8). In Lines 9-12, cliques $I_q = \{t_{10}, t_{11}, t_{12}\}$ are generated and added to Q_{new} (Line 11). In the next iteration, a new clique constraint $x_{10}+x_{11}+x_{12} \leq 1$ is added (Line 6), and the LP is solved again (Line 7), yielding $x_7 = x_{11} = 1$ and $x_5 = x_{10} = x_{12} = 0$. The loop ends when no variable has a value of 0.5 (Lines 2 to 13). The removal set $I_N = \{t_5, t_{10}, t_{12}\}$ is formed (Line 14), and redundancy is checked for minimality (Lines 16 to 18).

Example 10. Consider Figure 1a. Suppose the loss values computed by (8) are $L(t_5) = 0.005$, $L(t_7) = 1.220$, $L(t_{10}) = 0.813$, $L(t_{11}) = 1.213$, and $L(t_{12}) = 0.525$. For each conflict pair (t_i, t_l), retention probabilities are assigned proportionally to their loss values. For example, for (t_5, t_7), we have $P_{57} = \frac{0.005}{0.005+1.220} = 0.004$ and $P_{75} = 0.996$, so t_5 is removed with probability 0.996. Repeating this process for all conflict pairs yields an initial removal set $I'_N = \{t_5, t_{10}, t_{11}, t_{12}\}$. After sorting and minimality checking, t_{11} is reinserted, and the final removal set is $I_N = \{t_5, t_{10}, t_{12}\}$.